

CE2 Task 2

Intuition:

- Get all staff
- For each unique department, count how many staff in that dept
- Return the counts

After researching:

- Let MongoDB do the counting for me using \$group

```
db.collection('staff').aggregate([
  { $group: { _id: "$dept", count: { $sum: 1 } } }
])
```

- group all staff documents by their dept field
- for each group, sum up 1 for each document
- output is:

```
[{ "_id": "hr", "count": 2 }]
```

What I need to do:

(In models/staff.ts)

```
export async function countByDept(): Promise<{count: number, dept: string}[]> {
  if (!db) throw new Error("Database not initialized");
  const collection = db.collection(collectionName);
  const result = await collection.aggregate([
    { $group: { _id: "$dept", count: { $sum: 1 } } }
  ]).toArray();
  return result.map(r => ({ count: r.count, dept: r._id }));
}
```

(In routes/dept.ts)

```
router.get('/count', async function(req: Request, res: Response, next: NextFunction) {
  const counts = await staffmodel.countByDept();
  res.json(counts);
});
```

But this seems way too simple. In the ER diagram, we also see cardinality in play. From the lecture notes, this is the relationship in words:

- 1) For each entity object in STAFF, there exists only 1 corresponding entity object in DEPT.
- 2) For each entity object in DEPT, there exists many corresponding entity objects in STAFF.

We can enforce these relationships by:

- Ensuring that each STAFF must have 1 DEPT by making sure that they cannot have 0 DEPT (i.e. nonexistentdept must not occur)

(in routes/staff.ts)

```
router.get('/add/:id/:name/:code', async function(req: Request, res:
  Response, next: NextFunction) {
```

```

const dept = await deptmodel.find({ code: req.params.code });
if (dept.length === 0) {
  res.status(400).json({ error: `Department '${req.params.code}' does not exist` });
  return;
}
const staff = new staffmodel.Staff(req.params.id, req.params.name, req.params.code);
await staffmodel.insertMany([staff]);
res.json({ id: staff.id, name: staff.name, dept: staff.dept });
});

```

- Ensuring that each STAFF must have 1 DEPT by making sure they cannot have >1 DEPT (i.e. staff id can only appear once globally)

(in routes/staff.ts)

```

const existing = await staffmodel.find({ id: req.params.id });
if (existing.length > 0) {
  res.status(400).json({ error: `Staff ID '${req.params.id}' already exists` });
  return;
}

```

Then we output as so:

```

router.get('/add/:id/:name/:code', async function(req: Request, res: Response, next: NextFunction) {
  const dept = await deptmodel.find({ code: req.params.code });
  if (dept.length === 0) {
    res.status(400).json({ error: `Department '${req.params.code}' does not exist` });
    return;
  }
  const existing = await staffmodel.find({ id: req.params.id });
  if (existing.length > 0) {
    res.status(400).json({ error: `Staff ID '${req.params.id}' already exists` });
    return;
  }
  const staff = new staffmodel.Staff(req.params.id, req.params.name, req.params.code);
  await staffmodel.insertMany([staff]);
  res.json({ id: staff.id, name: staff.name, dept: staff.dept });
});

```

- Since MongoDB is schema-less, I can already insert as many STAFF in 1 DEPT so the second relationship is fulfilled by default.

Yay I'm done with Task 2!